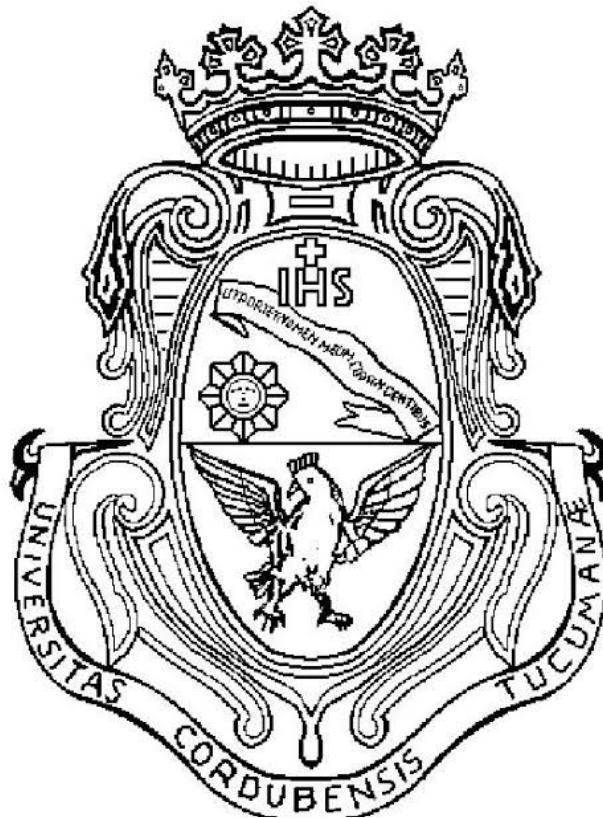


Redes de Computadoras

Informe Tarea 5

2026



Integrantes:

- Arias, Daniel Andrés
- Fernández y Fernández, Sergio Ezequiel
- Garzón, Pablo
- Gutierrez, Patricio
- Loza Denardi, Gabriel Jeremías
- Rocatagliatta, Leandro Agustin
- Zambellini, Matías Manuel

Introducción

En el presente trabajo se realizarán un breve repaso y los ejercicios del capítulo 20 del libro *Stallings - Comunicaciones y Redes de Computadores, 7.ª edición*, dedicado a los protocolos de transporte. El repaso incluirá el direccionamiento, la multiplexación, el esquema de créditos, los mecanismos de diálogo, las políticas de implementación de TCP y el protocolo UDP. Por su parte, para la ejercitación se seleccionaron once problemas que abarcan el control de flujo mediante créditos, el espacio de números de secuencia, el establecimiento y cierre de conexiones, la estimación del tiempo de ida y vuelta, el síndrome de la ventana absurda y el control de congestión de TCP.

Preguntas de repaso

20.1. ¿Qué elementos de direccionamiento son necesarios para especificar un usuario de servicio de transporte (TS) destino?

Para especificar de manera completa a un usuario del servicio de transporte (*Transport Service*) destino, es necesaria la siguiente información de direccionamiento:

- **Identificación del usuario:** Identifica el proceso o aplicación específica dentro de la estación de destino (representado habitualmente por el número de puerto).
- **Identificación de la entidad de transporte:** Especifica el protocolo o tipo de entidad de transporte utilizada (TCP o UDP, por ejemplo), lo cual es necesario si existe más de un tipo de entidad de transporte en la misma estación.
- **Dirección de la estación:** Identifica al dispositivo conectado a la red. En un entorno de redes interconectadas, corresponde a una dirección global de red (como la dirección IP).
- **Número de la red:** Identifica a la red específica dentro de un conjunto de redes interconectadas.

20.2. Describa cuatro estrategias por las que un usuario TS emisor pueda averiguar la dirección de un usuario TS receptor.

Para que un usuario del servicio de transporte emisor pueda averiguar la dirección de un usuario receptor, se establecen 4 estrategias divididas en dos estáticas y dos dinámicas:

Estrategias Estáticas

1. **Conocimiento previo de la dirección (Configuración del sistema):** El usuario TS conoce de antemano la dirección fija del usuario destino mediante la configuración del sistema. Se utiliza habitualmente para procesos de interés restringido o privado que no deben ser de acceso público.
2. **Direcciones bien conocidas (Well-Known Addresses):** A los servicios comunes o estandarizados de uso frecuente se les asigna una dirección fija y de conocimiento público. De este modo, cualquier usuario emisor puede conectarse directamente al puerto estándar del servicio deseado.

Estrategias Dinámicas

1. **Servidor de nombres (Name Server)**: El usuario TS emisor no conoce la dirección numérica exacta, sino un nombre genérico o global del servicio. El emisor consulta a un servidor de nombres, el cual busca en un directorio y le devuelve la dirección actual del usuario TS receptor.
2. **Creación de procesos bajo demanda**: Se utiliza cuando el proceso receptor no está ejecutándose de forma continua, sino que se crea al recibir la solicitud. El emisor envía una petición inicial a una dirección conocida donde escucha un proceso del sistema con privilegios. Este proceso del sistema genera la nueva aplicación receptora y le devuelve al emisor la dirección asignada al nuevo proceso para que pueda continuar la conexión.

20.3. Explique el uso de la multiplexación en el contexto de un protocolo de transporte.

La **multiplexación** cumple dos funciones fundamentales, según se analice respecto a las aplicaciones de capas superiores o respecto a los servicios de red subyacentes:

1. Multiplexación con las capas superiores (Usuarios de Transporte)

- Permite que múltiples usuarios o aplicaciones compartan la misma entidad del protocolo de transporte.
- La entidad de transporte distingue y encamina los datos de cada aplicación destino utilizando puntos de acceso al servicio (**SAP**) o números de puerto (al igual que en TCP o UDP).

2. Multiplexación con los servicios de la capa de red

En lo referido a las conexiones provistas por la red subyacente, la entidad de transporte implementa dos modalidades:

- **Multiplexación hacia arriba (ascendente)**: Consiste en agrupar múltiples conexiones de transporte sobre una única conexión de red. Se usa principalmente para optimizar costes y recursos.
- **Multiplexación hacia abajo (descendente)**: Consiste en dividir una única conexión de transporte entre múltiples conexiones de la capa de red subyacente. Se usa principalmente para incrementar el rendimiento, la velocidad de transferencia o la fiabilidad.

20.4. Describa brevemente el esquema de créditos utilizado por TCP para el control de flujo.

El esquema de **créditos** utilizado por TCP para el control de flujo se basa en los siguientes puntos clave:

1. **Desacoplamiento entre confirmaciones y control de flujo**: El esquema de créditos separa la confirmación de la recepción de datos de la concesión de nuevos créditos. Se pueden confirmar octetos recibidos sin necesidad de autorizar más espacio de emisión, o conceder crédito adicional sin confirmar nuevos datos.

2. **Orientado a octetos (bytes)**: A cada octeto individual de datos transmitido se le asigna un número de secuencia (**SN**) único. Por ello, los créditos y las ventanas se miden en número exacto de octetos.
3. **Mecanismo en la cabecera**: Cada segmento TCP incluye en su cabecera tres campos fundamentales para la regulación del flujo:
 - **SN (Sequence Number)**: Número de secuencia del primer octeto de datos transmitido en ese segmento.
 - **AN (Acknowledgment Number)**: Confirma acumulativamente que todos los octetos anteriores a AN se han recibido correctamente e indica que el receptor espera recibir el octeto número AN.
 - **Ventana (Window, W)**: Especifica el crédito otorgado, que es el número de octetos adicionales que se le autoriza a enviar al emisor a partir del octeto AN (comprendiendo el rango desde AN hasta $AN + W - 1$).
4. **Dinámica y gestión de la ventana**:
 - Cada vez que el emisor transmite datos, consume crédito y reduce el tamaño de su ventana de transmisión disponible.
 - Al recibir un segmento con un nuevo crédito (W), el emisor extiende o avanza el límite superior de su ventana.
 - Si la memoria temporal del receptor se satura, este puede fijar la ventana en cero ($W = 0$), lo que frena los envíos del emisor sin cerrar la conexión hasta que el receptor disponga de espacio libre y emita un nuevo crédito.
5. **Servicio al control de congestión**: Aunque fue diseñado para evitar que un emisor rápido desborde la memoria del receptor extremo a extremo, este esquema de crédito sirve también como base para que TCP controle dinámicamente el tráfico en la red e influya en los algoritmos de prevención de congestión (mediante ajustes en la ventana permitida).

20.5. ¿Cuál es la diferencia principal entre el esquema de créditos de TCP y el esquema de control de flujo de ventana deslizante utilizada por muchos otros protocolos, como por ejemplo HDLC?

La diferencia principal radica en el **desacoplamiento entre la confirmación de recepción y la concesión de control de flujo**.

Algunos aspectos clave que marcan esta diferencia son:

1. **Desacoplamiento de acuses**:
 - **HDLC (Ventana deslizante)**: La confirmación de una trama y la autorización para transmitir tramas adicionales van implícitamente unidas. Cuando el receptor envía una confirmación (**ACK**), la ventana de transmisión avanza automáticamente en una cantidad predeterminada.

- **TCP (Esquema de créditos):** Separa explícitamente la confirmación de la asignación de crédito. En la cabecera TCP se incluyen de forma independiente el número de confirmación (AN) y la ventana de crédito (W). De este modo, TCP puede confirmar segmentos recibidos sin otorgar crédito adicional, así como conceder crédito nuevo sin confirmar datos adicionales.
2. **Granularidad:**
 - **HDLC:** Mide los números de secuencia y la ventana en cantidad de tramas o bloques discretos.
 - **TCP:** Numera y gestiona el flujo a nivel de octetos (bytes) individuales. Cada octeto tiene su propio número de secuencia y el crédito especifica el rango exacto de bytes que se le permite enviar al emisor.
 3. **Flexibilidad:**
 - **HDLC:** Opera con una ventana de tamaño fijo.
 - **TCP:** Utiliza una ventana dinámica. El receptor ajusta continuamente la cantidad de crédito asignado (W) según la disponibilidad de sus memorias temporales, pudiendo fijar $W = 0$ para detener el tráfico del emisor cuando la memoria se satura y reabrirlo posteriormente.

20.6. Explique los mecanismos de diálogo en dos y tres pasos.

1. Diálogo en dos pasos (Two-Way Handshake)

- **Mecanismo:** La entidad de transporte emisora (A) inicia el establecimiento de la conexión enviando un segmento de solicitud de conexión (SYN) a la entidad receptora (B). Al recibirlo, B responde enviando a su vez un segmento SYN de confirmación/aceptación.
- **Limitación en redes no fiables:** Aunque es un esquema sencillo adecuado para redes fiables, falla en redes con conmutación de paquetes o con retardos variables.

2. Diálogo en tres pasos (Three-Way Handshake)

- **Mecanismo:** Usado para resolver la falta de fiabilidad de la red. En este esquema, cada extremo confirma explícitamente el SYN y el número de secuencia inicial (ISN, Initial Sequence Number) del otro:
 1. **Paso 1 (A to B):** A envía un segmento SYN con su número de secuencia inicial ($SN = i$).
 2. **Paso 2 (B to A):** B recibe la solicitud, pasa al estado SYN RECEIVED y responde enviando un segmento SYN con su propio número de secuencia ($SN = j$), confirmando acumulativamente el SYN de A con $AN = i + 1$.
 3. **Paso 3 (A to B):** A confirma la recepción del SYN de B enviando un acuse con $AN = j + 1$. Al completarse este paso, ambas entidades pasan al estado de conexión establecida (ESTAB).

20.7. ¿Cuál es el beneficio del mecanismo de diálogo en tres pasos?

El beneficio principal de este mecanismo es garantizar la apertura fiable de una conexión sobre una red no fiable, resolviendo así los problemas generados por paquetes duplicados o retrasados.

Sus principales ventajas son:

- **Prevención de conexiones "fantasma" o erróneas:** En redes con conmutación de paquetes donde los segmentos pueden retrasarse o duplicarse, un segmento de solicitud de conexión (SYN) obsoleto de una sesión anterior podría llegar a un receptor. En un diálogo de dos pasos, el receptor aceptaría automáticamente la solicitud y crearía una conexión no deseada. El diálogo en tres pasos exige una confirmación explícita bidireccional antes de pasar al estado de conexión establecida (ESTAB), impidiendo que peticiones viejas abran conexiones inválidas.
- **Sincronización mutua de los números de secuencia iniciales (ISN):** Permite que ambas entidades de transporte notifiquen y confirmen recíprocamente el número de secuencia con el que cada una comenzará a numerar sus datos. Asegurando así la entrega ordenada de la información en ambos sentidos de la comunicación full - duplex.
- **Mecanismo de cancelación mediante reinicio (RST):** Si un extremo recibe un SYN/ACK que no corresponde a ninguna solicitud de conexión activa por su parte (debido a la llegada de un SYN retrasado al receptor, por ejemplo), el emisor responde inmediatamente con un segmento de reinicio (RST). Esto fuerza al receptor a abortar la conexión errónea y liberar los recursos asignados en vano.

20.8. Defina las características de urgencia y forzado de TCP.

1. Flujo de datos forzado (Push)

- **Concepto:** Por omisión, TCP decide de forma autónoma cuándo ha acumulado suficientes datos en su memoria temporal para empaquetar y transmitir un segmento, así como cuándo entregar los datos recibidos a la aplicación de destino. El servicio de forzado permite al usuario requerir a TCP que transmita y entregue la información de inmediato sin esperar a completar un bloque o llenar la memoria temporal.
- **Mecanismo:** La entidad emisora activa el bit indicador **PSH** en la cabecera del segmento TCP. Al recibir un segmento con este bit activo, la entidad TCP receptora entrega inmediatamente los datos acumulados a la aplicación de destino.
- **Aplicación:** Funciona como una función de fin de bloque y resulta esencial en aplicaciones interactivas o ante interrupciones lógicas en los datos, donde la información debe transmitirse y entregarse sin demoras.

2. Señalización de datos urgentes (Urgent)

- **Concepto:** Proporciona un mecanismo para notificar al usuario o aplicación de destino que en el flujo de datos que está recibiendo existe información prioritaria o "urgente".
- **Mecanismo:** Los datos urgentes se transmiten dentro del flujo de datos ordinario. Para gestionarlos, TCP utiliza dos elementos de la cabecera:
 1. El bit indicador **URG**, que señala que el segmento incluye datos urgentes y que el campo de puntero es válido.
 2. El campo **Puntero urgente (Urgent Pointer)**, de 16 bits, cuyo valor, al sumarse al número de secuencia del segmento, especifica el número de secuencia del último octeto de la secuencia urgente. Esto le permite a la entidad receptora determinar la cantidad exacta de datos urgentes que están llegando.
- **Aplicación:** Al recibir la señalización, TCP alerta a la aplicación receptora de que existen datos urgentes disponibles. Es responsabilidad del usuario de destino determinar la acción concreta a realizar.

20.9. ¿Qué es una opción en los criterios de implementación de TCP?

Una **opción en los criterios de implementación de TCP** hace referencia a una decisión de diseño sobre aspectos del protocolo donde el estándar oficial no impone un algoritmo único, sino que deja libertad para seleccionar entre distintas alternativas o estrategias de funcionamiento.

Aspectos fundamentales:

- **Interoperabilidad garantizada:** El estándar TCP está diseñado para que dos entidades de transporte que elijan opciones de implementación distintas puedan comunicarse e interoperar sin problemas entre sí.
- **Impacto en el rendimiento:** Aunque la comunicación sea compatible, la elección de una u otra opción afecta directamente al rendimiento global (sobrecarga del procesador, latencia, número de interrupciones del sistema y nivel de tráfico suplementario en la red).

Algunos ejemplos de diseño con opciones de implementación son:

1. **Política de envío (Send policy):** Determina cuándo la entidad TCP emisora debe empaquetar los datos recibidos de la aplicación en un segmento para transmitirlos.
2. **Política de entrega (Deliver policy):** Define cuándo entregar los datos recibidos al usuario destino.
3. **Política de aceptación (Acceptance policy):** Establece el comportamiento cuando llegan segmentos desordenados fuera de secuencia.
4. **Política de retransmisión (Retransmission policy):** Define qué segmentos retransmitir al vencer un temporizador.
5. **Política de confirmación (Acknowledgment policy):** Establece cuándo emitir los acuses de recibo (ACK).

20.10. ¿Cómo puede utilizarse TCP para tratar la congestión de red o de interconexión de red?

Aunque el mecanismo de asignación de créditos mediante ventanas de TCP se diseñó originalmente para el control de flujo extremo a extremo (evitar saturar la memoria del receptor), este mismo esquema se utiliza para el control de congestión en la red o interconexión de redes.

Como TCP opera en los sistemas finales y no modifica los nodos internos de la red, detecta la presencia de congestión a través de dos señales implícitas: el **incremento en los tiempos de retardo de ida y vuelta (RTT)** y la **pérdida de segmentos**.

Para tratar la congestión, TCP actúa regulando su tráfico mediante dos componentes principales: la **gestión de la ventana** y la **gestión de los temporizadores de retransmisión**.

20.11. ¿Qué proporciona UDP que no ofrezca IP?

Debido a su naturaleza no orientada a conexión y no fiable, UDP realiza muy pocas tareas adicionales sobre las que ya proporciona IP.

UDP proporciona dos funciones principales que IP no ofrece:

- **Direccionamiento por puertos (Multiplexación/Demultiplexación de procesos)**: El protocolo IP direcciona únicamente estaciones o dispositivos finales en la red mediante direcciones IP de origen y destino. UDP añade a la cabecera los campos de **puerto origen** y **puerto destino**, lo que permite identificar la aplicación o proceso específico dentro del sistema destino que debe recibir la información y permite que múltiples aplicaciones compartan la capa de transporte.
- **Detección de errores en los datos de usuario**: La suma de comprobación (*checksum*) de IP se calcula únicamente sobre la cabecera IP, sin verificar los datos transportados. En cambio, UDP incluye un campo de **suma de comprobación opcional** que cubre todo el segmento UDP (cabecera + datos de usuario) junto con una pseudocabecera IP. Esto permite a la entidad receptora comprobar si el contenido del mensaje se ha corrompido durante la transmisión y descartarlo en caso de error.

Ejercicios

Ejercicio 20.3

Dos entidades de transporte se comunican a través de una red fiable. Supongamos que el tiempo normalizado para transmitir un segmento es igual a 1. Supongamos que el retardo de propagación extremo a extremo vale 3 y que la entrega de un segmento recibido al usuario de transporte requiere un tiempo de 2. El emisor tiene inicialmente concedido un crédito de siete segmentos. El receptor utiliza un criterio de control de flujo conservador y actualiza su asignación de créditos en cuanto puede. ¿Cuál es el máximo rendimiento alcanzable?

Resolución:

Con un criterio conservador, el receptor sólo concede crédito por el espacio de memoria que efectivamente tiene libre. Un espacio se libera cuando el segmento se entrega al usuario, y en ese momento el receptor devuelve un crédito, que tarda 3 unidades en llegar al emisor (consideramos despreciable el tiempo de transmisión de la confirmación).

Ciclo completo de un segmento, desde que empieza a transmitirse hasta que su crédito vuelve al emisor:

$$T_{\text{ciclo}} = t_{\text{transmisión}} + t_{\text{propagación}} + t_{\text{entrega}} + t_{\text{retorno}} = 1 + 3 + 2 + 3 = 9$$

Hay dos límites posibles para el rendimiento:

- **Límite impuesto por la ventana:** con 7 créditos, el emisor puede enviar como máximo 7 segmentos por ciclo: $\frac{7}{9} \approx 0,78$ segmentos por unidad de tiempo.
- **Límite impuesto por el receptor:** la entrega es secuencial y cada segmento demora 2, de modo que el receptor no puede liberar memoria más rápido que $\frac{1}{2} = 0,5$ segmentos por unidad de tiempo.

El rendimiento máximo es el menor de ambos. La evolución temporal de los primeros segmentos lo confirma:

Segmento	Fin de transmisión	Llegada a B	Fin de entrega	Crédito llega a A
1	1	4	6	9
2	2	5	8	11
3	3	6	10	13
4	4	7	12	15
7	7	10	18	21
8	10	13	20	23
9	12	15	22	25
10	14	17	24	27

El emisor agota sus 7 créditos en $t=7$ y queda esperando hasta $t=9$. A partir de allí los créditos vuelven al ritmo con el que el receptor entrega los segmentos (uno cada 2 unidades), por lo que el emisor transmite un segmento cada 2 unidades de tiempo.

Conclusión: el máximo rendimiento alcanzable es 0,5 segmentos por unidad de tiempo, es decir, el 50% de la capacidad de transmisión. El cuello de botella es la velocidad de entrega del receptor; aumentar el crédito no mejoraría el rendimiento, ya que en régimen estable sólo se usan $0,5 \times 9 = 4,5$ de los 7 créditos disponibles.

Ejercicio 20.5

Con un servicio de red fiable y ordenado, ¿son estrictamente necesarios los números de secuencia de los segmentos? ¿Qué capacidad se pierde sin ellos?

Resolución:

No son estrictamente necesarios. Si la red garantiza que todos los segmentos llegan, en orden y sin duplicados, el receptor no necesita detectar pérdidas, reordenar ni descartar duplicados: ambos extremos pueden llevar un contador implícito de los datos enviados y recibidos, y el control de flujo puede realizarse con créditos incrementales del tipo «puede enviar k segmentos más».

Sin embargo, se pierden algunas capacidades:

- **Desacoplar confirmación y crédito de forma explícita:** en el esquema de créditos, el par (AN, W) se refiere a posiciones concretas del flujo de datos. Sin números de secuencia, el crédito sólo puede expresarse como un incremento relativo, y si alguna vez los contadores de ambos extremos se desincronizan no hay forma de volver a sincronizarlos.
- **Recuperarse ante un reinicio o fallo de la red:** después de un reinicio, los extremos no pueden saber qué datos llegaron y cuáles deben retransmitirse (ver el problema planteado en el ejercicio 20.6).
- **Referenciar posiciones del flujo:** mecanismos como el cierre ordenado (indicar cuál fue el último dato enviado) o la señalización de datos urgentes necesitan identificar octetos concretos del flujo.

Ejercicio 20.9

En una red que tiene un tamaño máximo de paquete de 128 bytes, un tiempo de vida máximo de 30 s y un número de secuencia de paquetes de 8 bits, ¿cuál es la máxima tasa de transmisión de datos por conexión?

Resolución:

Un número de secuencia no puede reutilizarse mientras exista en la red un paquete anterior que lo lleve; si no, un duplicado retrasado podría confundirse con un paquete nuevo (Figura 20.5 del libro). Por lo tanto, en un intervalo igual al tiempo de vida máximo se pueden enviar como máximo tantos paquetes como números de secuencia distintos haya.

Números de secuencia disponibles:

$$2^8 = 256$$

Datos máximos enviados durante un tiempo de vida:

$$256 \times 128 \text{ bytes} \times 8 \text{ bits/byte} = 262.144 \text{ bits}$$

Tasa máxima por conexión:

$$R_{\max} = \frac{262.144 \text{ bits}}{30 \text{ s}} \approx 8.738 \text{ bps} \approx 8,74 \text{ kbps}$$

Conclusión: la tasa queda limitada a unos 8,7 kbps por conexión. Para transmitir más rápido haría falta un espacio de números de secuencia mayor (cada bit adicional duplica la tasa máxima) o un tiempo de vida de paquete menor.

Ejercicio 20.10

¿Es posible que se produzca un bloqueo mutuo utilizando un diálogo en dos pasos en lugar de un diálogo en tres pasos? Dé un ejemplo o demuéstrelo en caso contrario.

Resolución:

Sí, es posible. Tomando como base la situación de la Figura 20.7 del libro:

1. Una conexión anterior entre A y B ya se cerró, pero un segmento **SYN** i obsoleto de esa conexión sigue circulando por la red.
2. El SYN i llega a B. Como en el diálogo en dos pasos nada permite reconocerlo como viejo, B lo toma como una solicitud nueva, responde con **SYN** j y considera la conexión abierta, esperando datos a partir del número de secuencia i .
3. Mientras tanto, A decide abrir una nueva conexión y envía **SYN** k . B lo descarta porque cree que ya tiene una conexión abierta con A (lo trata como un duplicado).
4. A recibe el SYN j y lo interpreta como la respuesta a su SYN k , de modo que también considera la conexión abierta.
5. A envía datos numerados a partir de k . B los rechaza porque espera la secuencia i , por lo que nunca los confirma ni concede créditos. A vuelve a retransmitir cada vez que vence su temporizador, y B sigue esperando segmentos que nunca llegarán.

Ambas entidades creen tener una conexión válida, pero ninguna puede avanzar: A espera una confirmación que nunca llega y B espera datos que nunca recibe. El diálogo en tres pasos evita esta situación porque cada extremo debe confirmar explícitamente el SYN y el número de secuencia inicial del otro: en el paso 4, A recibiría un SYN que confirma i en lugar de k , lo detectaría como erróneo y respondería con un RST.

Ejercicio 20.14

En la discusión sobre el cierre de la conexión con referencia a la Figura 20.8, se estableció que además de recibir una confirmación de su segmento FIN y enviar una confirmación del segmento FIN recibido, una entidad TCP debe esperar un intervalo de tiempo igual al doble del máximo tiempo de vida esperado de un segmento (el estado TIME WAIT). La recepción de un ACK de su segmento FIN le asegura que todos los segmentos que ha enviado han sido recibidos por el otro extremo. El envío de un ACK del segmento FIN del otro extremo asegura a la otra entidad que todos sus segmentos han sido recibidos. Dé una razón por la que se necesite aún esperar antes de cerrar la conexión.

Resolución:

La razón principal es que **el último ACK puede perderse**. Ese ACK, que confirma el FIN del otro extremo, no recibe confirmación. Si se pierde, el otro extremo no sabrá que su FIN llegó y, al vencer su temporizador, lo retransmitirá. Si la entidad ya hubiese cerrado la conexión y borrado su estado, no reconocería ese FIN retransmitido: respondería con un RST y el otro extremo no podría cerrar de forma ordenada. Permaneciendo en TIME WAIT, la entidad puede volver a confirmar el FIN.

El tiempo de espera de $2 \times \text{MSL}$ (*Maximum Segment Lifetime*) cubre el peor caso de un viaje de ida (el ACK perdido) más uno de vuelta (el FIN retransmitido).

Una segunda razón es dejar que **desaparezcan de la red los segmentos retrasados** de la conexión. Si inmediatamente se abriera una nueva conexión entre los mismos puertos, un segmento duplicado y retrasado de la conexión anterior podría aceptarse como válido en la nueva. Al esperar $2 \times \text{MSL}$ se garantiza que cualquier segmento de la conexión antigua ya expiró.

Ejercicio 20.15

Normalmente, el campo «ventana» de la cabecera TCP da una asignación de créditos en octetos. Cuando se utiliza la opción de «escalado de ventana», el valor del campo «ventana» se multiplica por 2^F , donde F es el valor de la opción de escalado de ventana. El valor máximo de F que acepta TCP es 14. ¿Por qué se limita esta opción a 14?

Resolución:

El campo ventana tiene 16 bits, por lo que con el escalado máximo la ventana puede alcanzar:

$$W_{\max} = 2^{16} \times 2^{14} = 2^{30} \text{ octetos} \approx 1 \text{ GB}$$

El límite está dado por el espacio de números de secuencia, que en TCP es de 32 bits (2^{32} valores) y se recorre de forma cíclica. Para decidir si un segmento que llega es nuevo o es un duplicado antiguo, TCP compara números de secuencia en aritmética módulo 2^{32} , y esta comparación sólo es correcta si la distancia entre los números comparados es menor que la mitad del espacio, es decir 2^{31} .

En una conexión conviven la ventana del emisor y la del receptor, que pueden estar desfasadas entre sí hasta el tamaño de una ventana. Para que nunca haya ambigüedad, la suma de ambas debe ser como máximo 2^{31} :

$$2 \times W_{\max} \leq 2^{31} \implies W_{\max} \leq 2^{30}$$

Con $F=15$ la ventana podría llegar a 2^{31} y ya no se podrían distinguir los segmentos nuevos de los duplicados viejos. Por eso el escalado se limita a $F=14$.

Ejercicio 20.16

La elección de un valor inicial del estimador original de SRTT de TCP constituye un problema. En ausencia de alguna información especial sobre las condiciones de la red, la opción habitual es la de elegir un valor arbitrario, como 3 segundos, y esperar que converja rápidamente a un valor preciso. Si la estimación es demasiado baja, TCP llevará a cabo retransmisiones innecesarias. Si es demasiado alta, TCP esperará demasiado tiempo antes de retransmitir en caso de que el primer segmento se pierda. Es más, la convergencia puede ser lenta, como indica este problema.

- a) Elija $\alpha=0,85$ y $\text{SRTT}(0)=3$ segundos, suponga que todos los valores de RTT medidos son iguales a 1 segundo y que no se producen pérdidas de paquetes. ¿Cuál es el valor de $\text{SRTT}(19)$?

- b) Sea ahora $SRTT(0)=1$ segundo y suponga que los valores medidos de RTT son 3 segundos y que no se produce pérdida de paquetes. ¿Cuál es el valor de $SRTT(19)$?

Resolución:

La Ecuación (20.3) define el promediado exponencial:

$$SRTT(K+1) = \alpha \cdot SRTT(K) + (1-\alpha) \cdot RTT(K+1)$$

Si todas las muestras de RTT son iguales, al desarrollar la recurrencia n veces se obtiene una suma geométrica:

$$SRTT(n) = \alpha^n \cdot SRTT(0) + (1-\alpha) RTT \cdot \frac{1-\alpha^n}{1-\alpha} = \alpha^n \cdot SRTT(0) + (1-\alpha^n) \cdot RTT$$

Para $n=19$:

$$\alpha^{19} = 0,85^{19} \approx 0,0456$$

- a) $SRTT(0)=3s$, $RTT=1s$:

$$SRTT(19) = 0,0456 \cdot 3 + (1-0,0456) \cdot 1 = 1 + 2 \cdot 0,0456 \approx 1,091s$$

- b) $SRTT(0)=1s$, $RTT=3s$:

$$SRTT(19) = 0,0456 \cdot 1 + (1-0,0456) \cdot 3 = 3 - 2 \cdot 0,0456 \approx 2,909s$$

Conclusión: en ambos casos, después de 19 muestras el estimador todavía arrastra un error de unos 0,09s (un 4,6% del error inicial de 2 segundos). Con $\alpha=0,85$ cada muestra corrige sólo el 15% del error, por lo que una mala elección de $SRTT(0)$ tarda muchas muestras en olvidarse.

Ejercicio 20.17

Una mala implementación del esquema de ventana deslizante de TCP puede llevar a un rendimiento extremadamente malo. Existe un fenómeno conocido como el «síndrome de la ventana absurda» (SWS, *Silly Window Syndrome*), que puede fácilmente causar una degradación del rendimiento en varios factores de 10. Como ejemplo de SWS, considere una aplicación que está ocupada en la transferencia de un fichero largo y que TCP está transfiriendo el fichero en segmentos de 200 octetos. El receptor inicialmente asigna un crédito de 1.000. El emisor agota esta ventana con 5 segmentos de 200 octetos. Ahora suponga que el receptor devuelve una confirmación por cada segmento y proporciona un crédito adicional de 200 octetos por cada segmento recibido. Desde el punto de vista del receptor, esto abre la ventana de nuevo a 1.000 octetos. Sin embargo, desde el punto de vista del emisor, si la primera confirmación llega tras haber enviado cinco segmentos, se dispone de una ventana de sólo 200 octetos. Suponga que en algún momento el receptor calcula una ventana de 200 octetos pero tiene sólo 50 octetos para enviar hasta llegar a un punto de forzado. Por tanto, envía 50 octetos en un segmento, seguido de 150 octetos en el siguiente segmento, y reanuda la transmisión de segmentos de 200 octetos. ¿Qué podría ahora ocurrir para dar lugar a un problema de rendimiento? Plantee el SWS en términos más generales.

Resolución:

Como el receptor confirma cada segmento por separado y concede exactamente el crédito correspondiente a lo que recibió, el segmento de 50 octetos genera un crédito de 50 octetos y el de 150 uno de 150. El emisor, que transmite en cuanto tiene crédito, usa esos créditos tal como llegan: envía un segmento de 50 octetos al recibir el crédito de 50 y uno de 150 al recibir el de 150. Esos segmentos vuelven a generar créditos de 50 y 150, y así sucesivamente.

La fragmentación de la ventana se **perpetúa**: los pedazos pequeños nunca vuelven a juntarse en segmentos de 200 octetos. Si vuelve a ocurrir algo parecido (otro punto de forzado, una aplicación que lee pocos datos), la ventana se parte en trozos cada vez más chicos, hasta llegar a segmentos de muy pocos octetos. Como cada segmento lleva al menos 40 octetos de cabecera (20 de TCP y 20 de IP), con un segmento de 50 octetos de datos casi la mitad de lo transmitido es cabecera, y además se multiplican las confirmaciones y el procesamiento por segmento.

En términos generales, el SWS ocurre cuando el flujo de datos se va dividiendo en segmentos muy pequeños, ya sea porque el **receptor** anuncia ventanas pequeñas apenas libera un poco de memoria, o porque el **emisor** envía segmentos pequeños apenas tiene un poco de crédito o de datos. Una vez que la ventana se divide en pedazos pequeños, el esquema de créditos tiende a mantenerlos así, y el rendimiento cae porque se transporta muy poca información útil por cada segmento.

Ejercicio 20.18

TCP impone que tanto el receptor como el emisor incorporen mecanismos para hacer frente al SWS.

- Sugiera una estrategia para el receptor. Sugerencia: permita al receptor «mentir» sobre la capacidad de memoria temporal de que dispone bajo ciertas circunstancias. Plantee una regla razonable experimental para esto.
- Sugiera una estrategia para el emisor. Sugerencia: considere la relación entre la ventana máxima posible de envío y lo que hay disponible para enviar.

Resolución:

a) Estrategia del receptor: el receptor no debe anunciar aumentos pequeños de la ventana. Aunque tenga algo de memoria libre, sigue anunciando una ventana cero (o no amplía la ventana anunciada) hasta que el espacio disponible sea suficientemente grande. Una regla razonable (propuesta por Clark) es abrir la ventana sólo cuando el espacio libre alcance:

$$\text{espacio libre} \geq \min \left(\text{MSS}, \frac{\text{memoria total}}{2} \right)$$

donde MSS es el tamaño máximo de segmento. Así, el emisor siempre recibe créditos que le permiten enviar segmentos grandes.

b) Estrategia del emisor: el emisor no debe transmitir un segmento sólo porque tiene un poco de crédito o pocos datos. Conviene que retenga los datos hasta que se cumpla alguna de estas condiciones:

- Puede enviar un segmento de tamaño completo (MSS).
- Puede enviar al menos la mitad de la ventana máxima que el receptor llegó a anunciar.
- No tiene datos pendientes de confirmación, o la aplicación pidió un envío forzado (*push*). En este caso se envía lo que haya.

Esta idea es la base del **algoritmo de Nagle**: mientras haya datos sin confirmar, los datos pequeños se van acumulando y se envían todos juntos cuando llega la siguiente confirmación.

Ejercicio 20.20

Una entidad TCP abre una conexión y utiliza el arranque lento. Aproximadamente, ¿cuántos tiempos de ida y vuelta se necesitan antes de que TCP pueda enviar N segmentos?

Resolución:

En el arranque lento, $cwnd$ empieza en 1 y se incrementa en 1 por cada confirmación recibida, de modo que se duplica en cada tiempo de ida y vuelta:

RTT	Segmentos enviados en ese RTT	Total acumulado
1	1	1
2	2	3
3	4	7
k	2^{k-1}	$2^k - 1$

Para haber enviado N segmentos hace falta que:

$$2^k - 1 \geq N \implies k = \lceil \log_2(N+1) \rceil$$

Conclusión: se necesitan aproximadamente $\log_2 N$ tiempos de ida y vuelta (suponiendo que el crédito del receptor no es el límite). Por ejemplo, para enviar 1.000 segmentos alcanzan unos $\lceil \log_2 1.001 \rceil = 10$ RTT. El crecimiento es exponencial, a pesar del nombre «arranque lento».

Ejercicio 20.21

Aunque el arranque lento con supresión de congestión es una técnica efectiva para hacer frente a la congestión, puede traducirse en largos tiempos de recuperación en redes de alta velocidad, como demuestra este problema:

- Suponga un retardo de ida y vuelta de 60 ms (lo que podría ocurrir a través de un continente), un enlace con un ancho de banda disponible de 1 Gbps y un tamaño de segmento de 576 octetos. Determine el tamaño de ventana necesario para



mantener lleno el cauce y el tiempo que tardaría en alcanzar el tamaño de ventana después de la expiración del temporizador utilizando el criterio de Jacobson.

b) Repita (a) para un tamaño de ventana de 16 kbytes.

Resolución:

Según el criterio de Jacobson, cuando vence un temporizador:

1. Se fija $ssthresh = cwnd/2$.
2. Se hace $cwnd = 1$ y se aplica arranque lento ($cwnd$ se duplica en cada RTT) hasta llegar a $ssthresh$.
3. A partir de $ssthresh$, $cwnd$ crece en 1 segmento por cada RTT.

a) Para mantener lleno el cauce, la ventana debe cubrir el producto ancho de banda por retardo:

$$W = \frac{1 \times 10^9 \text{ bps} \times 0,06 \text{ s}}{576 \times 8 \text{ bits/segmento}} = \frac{6 \times 10^7}{4.608} \approx 13.021 \text{ segmentos}$$

Al vencer el temporizador, $ssthresh = 13.021/2 \approx 6.510$ segmentos.

Fase de arranque lento (de 1 a 6.510 segmentos):

$$\lceil \log_2 6.510 \rceil = \lceil 12,67 \rceil = 13 \text{ RTT}$$

Fase lineal (de 6.510 a 13.021 segmentos, un segmento por RTT):

$$13.021 - 6.510 = 6.511 \text{ RTT}$$

Tiempo total de recuperación:

$$T = (13 + 6.511) \times 60 \text{ ms} = 6.524 \times 0,06 \text{ s} \approx 391 \text{ s} \approx 6,5 \text{ minutos}$$

b) Con una ventana de 16 kbytes ($16 \times 1.024 = 16.384$ octetos):

$$W = \frac{16.384}{576} \approx 28 \text{ segmentos}$$

Al vencer el temporizador, $ssthresh = 14$ segmentos.

Arranque lento (de 1 a 14): $\lceil \log_2 14 \rceil = 4 \text{ RTT}$. Fase lineal (de 14 a 28): 14 RTT.

$$T = (4 + 14) \times 60 \text{ ms} = 18 \times 0,06 \text{ s} \approx 1,08 \text{ s}$$

Sin embargo, con esa ventana el enlace queda muy subutilizado, ya que el rendimiento máximo es:

$$R = \frac{16.384 \times 8 \text{ bits}}{0,06 \text{ s}} \approx 2,18 \text{ Mbps}$$

lo que representa apenas el 0,2% del gigabit disponible.

Caso	Ventana	Tiempo de recuperación	Rendimiento máximo
a) Cauce lleno	≈ 13.021 segmentos	≈ 391 s	1 Gbps
b) 16 kbytes	≈ 28 segmentos	$\approx 1,08$ s	$\approx 2,18$ Mbps

Conclusión: en redes de alta velocidad y gran retardo, la fase de crecimiento lineal hace que, tras una sola pérdida, TCP tarde varios minutos en volver a aprovechar el enlace. Con una ventana pequeña la recuperación es rápida, pero porque nunca se usa más que una mínima parte de la capacidad.